

Secure Multi-Protocol Network & Security Suite

Student ID: 6788148

Project Category: Network Security & Secure Software Development

Overview

This project is an advanced evolution of low-level networking concepts into a functional security suite. It demonstrates mastery over the **OSI Model (Layer 4 - Transport)** by implementing both stateful (TCP) and stateless (UDP) communication. The suite has been engineered to transition from a simple data-exchange tool into a **hardened, encrypted, and logged** Command-Based System, complete with a diagnostic network scanner.

Features

- **TCP Port Scanner:** A reconnaissance tool that identifies open "doors" on a target IP using the TCP Connect method.
- **Encrypted Communication:** Implements **AES-128 bit encryption** via the Fernet (Symmetric) standard to ensure data confidentiality.
- **AAA Framework:** Basic implementation of **Authentication** (User/Pass login), **Authorization** (Command access), and **Accounting** (Detailed logging).
- **Dual-Protocol Support:** Custom-built TCP and UDP clients/servers optimized for different use cases (Reliability vs. Speed).
- **Automated Audit Logging:** Generates persistent `server_logs.txt` containing timestamps, client IPs, and full request/response history for forensic analysis.

Technical Implementation

1. The Logic: TCP Connect Scan

The scanner operates by attempting to finalize a **Three-Way Handshake** (SYN, SYN-ACK, ACK). By using `socket.connect_ex()`, the program captures the return code from the OS stack:

- **Code 0:** Connection successful (Port is Open).
- **Code 111/10061:** Connection refused (Port is Closed).

2. Security Hardening

Unlike standard socket scripts, this suite includes:

- **Input Sanitization:** Prevents server crashes by validating data types before processing math operations.
- **Symmetric Encryption:** Uses a shared secret key to transform plaintext commands into ciphertext, protecting against packet sniffing.
- **Timeout Management:** Prevents **Resource Exhaustion** by closing hanging connections after a 1-second threshold.

Core Components & Usage

A. The Network Scanner (`scanner.py`)

Used to map the attack surface of a target machine.

Python

None

```
import socket

target = "127.0.0.1"
for port in range(1, 1025):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(0.1)
    if s.connect_ex((target, port)) == 0:
        service = socket.getservbyport(port) if port < 1024 else
"Unknown"
        print(f"[OPEN] {port} ({service})")
    s.close()
```

B. Secure TCP System (Encrypted & Logged)

The server now acts as a secure gateway, requiring a handshake and key-matching before executing commands.

Sample Log Entry (`server_logs.txt`):

```
[2026-05-15 20:15:02] IP: 127.0.0.1 | REQ: MUL 10 5 | RES: 50 | AUTH: SUCCESS
```

Project Structure & Original Source

The foundation of this suite is built upon the following protocol implementations:

TCP Client-Server (Stateful/Reliable)

Used for the **Secure Command System** where data integrity is mandatory.

Python

```
# Server Logic Extract
server.listen(3)
conn, addr = server.accept()
data = conn.recv(1024).decode() # Receives structured commands
```

UDP Client-Server (Stateless/Fast)

Used for high-speed transmission where the overhead of a handshake is not required.

Python

```
# Client Logic Extract
client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
client.sendto(data.encode(), ("localhost", 1112)) # Fire-and-forget
```

TCP Server Code

Python

```
import socket

import datetime

# Configuration

HOST = "localhost"

PORT = 42000

def log_to_file(client_ip, request, result):

    timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    with open("server_logs.txt", "a") as f:

        log_entry = f"[{timestamp}] IP: {client_ip} | REQ: {request} | RES: {result}\n"
```

```

f.write(log_entry)

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

server.bind((HOST, PORT))

server.listen(5)

print(f"[*] Secure Server listening on {PORT}...")

while True:

    conn, addr = server.accept()

    try:

        data = conn.recv(1024).decode().strip()

        if not data: continue

        # STEP 1 & 3: Structured Commands (e.g., "ADD 10 20")

        parts = data.split()

        command = parts[0].upper()

        x = int(parts[1])

        y = int(parts[2])

        # STEP 4: Logic & Error Handling

        if command == "ADD": result = x + y

        elif command == "SUB": result = x - y

        elif command == "MUL": result = x * y

        elif command == "DIV":

            result = x / y if y != 0 else "Error: Div by Zero"

        else:

            result = "Error: Unknown Command"

        # STEP 5: Logging

```

```

log_to_file(addr[0], data, result)

print(f"StudentID: 6788148 | Handled {command} for {addr}")

conn.send(str(result).encode())

except Exception as e:

    error_msg = f"Error: {str(e)}"

    print(f"!! {error_msg}")

    conn.send(error_msg.encode())

finally:

    conn.close()

```

TCP Client Code

```

Python
import socket

import time

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

client.connect(("localhost", 42000))

print("Commands: ADD, SUB, MUL, DIV (Example: MUL 5 10)")

cmd = input("Enter Command and two numbers: ")

client.send(cmd.encode())

result = client.recv(1024).decode()

print("-" * 30)

print(f"StudentID: 6788148")

print(f"Time: {time.asctime()}")

```

```
print(f"Server Response: {result}")
```

```
print("-" * 30)
```

```
client.close()
```